COMP3711 Midterm Cheat Sheet v2 — SIDE A: core theory — Fri Oct 16, 20:00–22:00, LTC (L2). Coverage: prerequisites → binary search → mergesort/inversions → max subarray → multiplication → randomization → quicksort/selection → heaps → lower bounds & linear sorting. Answer = idea · pseudocode (define I/O) · running time (brief). No correctness proofs needed.

A1. Asymptotics

f=O(g): $\exists c,n_0$: f(n)≤c·g(n) ∀n≥n_0 (lim sup f/g<∞). **Ω**: f≥c·g (lim inf f/g>0). **Θ**: both (prove each separately). **o**: f/g→0. Worst-case Θ(g) = O(g) on all inputs + **an input family (one per n)** with Ω(g). Properties: transitive; O+O=O; max(f,g)=Θ(f+g) (max ≤ f+g ≤ 2max). Σ of k=O(f) terms is O(f) only for **fixed** k (T_i=i·n: g_n = Θ(n^3)). T_1,T_2=O(f) ⇏ T_1/T_2=O(1) or T_1=O(T_2).

Ladder: 1 < loglog n < log n < (log n)^c < n^ε (any ε>0) < n < n log n < n^1.5 < n² < n^c < 2^(√n) < 2^n < 3^n < n! < n^n. Polylog < any polynomial; polynomial < any exponential c^n (c>1). 10^(10^10) is Θ(1).

Rules: log base irrelevant. log(n^c)=Θ(log n). a^(log_b n) = n^(log_b a): $3^{(\log_2 n)}$=n^1.585, $4^{(\log_2 n)}$=n², $16^{(\log_4 n)}$=n², $3^{(\log_4 n)}$=n^0.79, 2^(loglog n)=log n. Exponent: **added** const same class (2^(n+5)=Θ(2^n)); **multiplied** const different (2^(2n)=4^n ≠ O(2^n); 5^n=o(7^n); √(4^n)=2^n; 2^n vs 2^(n/2): set m=2^(n/2), m² vs m). 2^f=Θ(2^g) iff lf–gl=O(1) (f=Θ(g) is NOT enough). n! = Θ(√(n/e)^n); log(n!) = Θ(n log n); n! ≥ (n/2)^(n/2); n!/(n–2)! = n²–n. n^(1/log n)=2. (log n)^(log n) = n^(loglog n) = 2^(log n·loglog n); n^(√log n)=2^((log n)^1.5); 2^((log n)²). **Rank weird functions: rewrite every one as 2^(h(n)) and compare the exponents h.** Substitute m=log n or m=√n (√log n vs log √n = ½log n: Ω only). log(2^n)=n=2^(log n).

Sums: Σ_{i≤n} i^c = Θ(n^(c+1)) (upper: n·max term; lower **half trick**: top half, each ≥ (n/2)^c). Σi = n(n+1)/2. Σ i·log i = Θ(n² log n). Σ √i·n = Θ(n^2.5); Σ i√i = Θ(n^2.5) vs n^(4/3): B wins. **H_n = Σ1/i = Θ(log n)** (doubling blocks, each block in [½,1], log n blocks). Σ_{i≥0} r^i: r<1 → Θ(1) (=1/(1−r)); r>1 → Θ(r^k) last term; r=1 → k+1. Σ_{i≥0} 2^(−i) = 2 = Θ(1). Σ i/2^i = 2; Σ_{k≥1} k x^(k−1) = 1/(1−x)²; Σ k/2^(k+1) ≤ 1. Σ_{i=k+1}^{N} 1/i = ln(N/k)+O(1). (1−1/n)^n → 1/e ≈ 0.37; (1+1/n)^n → e. Triple loop i≤j, k∈[i,j]: Θ(n³).

A / B / U table

Reduce to dominant term, logs base 2. **U** iff a one-sided bound (only O or only Ω) leaves the order undecided (O(n²) vs Θ(n log n): U; Ω(n³) vs O(n^2.81): B; O(polylog) vs Ω(n^0.1): A — the bound still forces it; O(log n) vs Θ(loglog n): U; O vs O: U; Ω vs Ω: U). Same Θ class → U ("always faster" fails). Σ-expressions and recurrences: solve first (4T(n/4)+n = n log n; 5T(n/2)+n² = n^2.32; 2T(n/2)+n = n log n; 3T(n/2)+n = n^1.585). $\log_3$(n!) = Θ(n log n); (log n!)³ = Θ(n³ log³ n): read sub vs superscript carefully.

A2. Recurrences

Master (T=aT(n/b)+n^d): a>b^d → Θ(n^(log_b a)) leaves; a=b^d → Θ(n^d log n); a<b^d → Θ(n^d) root. Extra log in f: same case ⇒ one more log power (4T(n/4)+n log n = n log² n). **Leaves = a^(log_b n) = n^(log_b a).** Course form: T=aT(n/b)+n: a>b → n^(log_b a); a=b → n log n; a<b → n.

Expansion (write this):
T(n)=aT(n/b)+f(n)=a²T(n/b²)+a f(n/b)+f(n)=…=a^k T(n/b^k)+Σ_{i<k} a^i f(n/b^i). Stop k=log_b n. Level i cost a^i f(n/b^i): same each level → ×log_b n; geometric ↓ → Θ(f(n)); geometric ↑ → Θ(last)=Θ(leaves). **Tree table:** level i: a^i nodes, size n/b^i, cost/node f(n/b^i), level total; leaves n^(log_b a)·T(1). **√n:** n=2^m, S(m)=T(2^m): T(√n)→S(m/2). **Induction:** guess, prove T(n)≤cn^α with the same c. T(n−1) type: expand n−1 levels.

T(n)=	Θ	T(n)=	Θ
T(n/2)+1	log n	2T(n/2)+n	n log n
2T(n/2)+√n	n (leaves)	2T(n/2)+n log n	n log² n
3T(n/5)+2n	n	3T(n/4)+n (or +n²)	n (n²)
3T(n/3)+n	n log n	3T(n/2)+n	n^1.585
4T(n/2)+n	n²	4T(n/2)+n²	n² log n
7T(n/2)+n²	n^2.81	8T(n/2)+n²	n³
64T(n/4)+n³	n³ log n	27T(n/3)+n²	n³
9T(n/3)+n²	n² log n	10T(n/3)+n²	n^($\log_3$10)≈n^2.1
4T(n/4)+n $\log_4$n	n log² n	2T(n/4)+√n	√n log n
2T(n/4)+n	n	T(n/8)+1	log n
2T(√n)+log n	log n·loglog n	T(√n)+log n	log n
T(n/2)+log n	log² n	T(n/2)+n	n (=2n−1)
T(n−1)+n	n²	2T(n−1)+1	2^n (=2^n−1)
T(αn)+T(βn)+n, α+β=1	n log n	same, α+β<1	n
T(n/4)+T(3n/4)+n	n log n	T(n/4)+T(3n/4)+1	n
5T(n/3)+n (Toom-3)	n^1.465	7T(n/4)+n	n^1.404

A3. Binary search & D&C designs

```
BS(A,l,r,x): if l>r nil; m=⌊(l+r)/2⌋;
A[m]=x→m;
  x<A[m]→BS(l,m-1) else BS(m+1,r)
T=T(n/2)+2=2log n+1
LowerBound(x): l=1,r=n+1; while l<r:
m=⌊(l+r)/2⌋;
  if A[m]<x l=m+1 else r=m; return l
(first ≥x; "≤" → first >x
  = successor; n+1 if none)   count(x)=
first>x − first≥x
Rotated (A[k] max): A[m]>A[m+1]→m;
A[m]≥A[l]→right else left;
  search x: x≥A[1]→BS(1,k) else
BS(k+1,n).  Last 0 in 0…01…1:
  A[m]=0&A[m+1]=1→m; A[m]=0→right else
left; k≪n: gallop i=1,2,4..
  until A[i]=1 then FindZ(i/2,i): O(log
k).  Bitonic peak: A[m]<A[m+1]→right.
LastOcc(A,p,r,x) [A[p]=x]: if p=r or
A[r]=x ret r; q=mid;
  A[q]=x&A[q+1]>x→q; A[q]=x→Last(q+1,r);
else Last(p,q-1)
Distinct values (Θ(log n) many): repeat
LastOcc from each
  block start, stop when it returns n →
O(log² n).
```

k-th smallest of 2 sorted arrays O(log k): compare A[k/2], B[k/2]; the smaller one + its prefix have rank<k → discard k/2 from that side; recurse with k/2 (missing entry = ∞); k=1: min of heads. (Alternatives: merge until k: Θ(k); binary search on split i: O(log min(m,k)).)

Template: Name(input)→output; base case; split; recurse; combine (say exactly); T(n)=aT(n/b)+split+combine; solve; one line why combine is right. **Hints in target bound:** log²n ⇒ log n binary searches; log k ⇒ halve k; n log log n ⇒ blocks of size log n; k log k ⇒ heap of size k; n expected ⇒ random pivot / good-pivot stages; n+k log k ⇒ linear pass then sort k items; n log k ⇒ k-way heap or recursion depth log k; n log m with Select+Split ⇒ T(n,m)=2T(n/2,m/2)+n, T(n,1)=n.

Mergesort 2T(n/2)+n = n log n + n (every input is worst case ⇒ Θ); merge: two pointers, ∞ sentinels, ties take left (stable), Θ(n) always. **Inversions**: during merge, when R[j] is placed add (m−l−i+2) = #remaining left items; Θ(n log n); naive n²; expected inversions of random perm = n(n−1)/4 (pair inverted w.p. ½); insertion sort = Θ(n+I). **Max subarray**: n³ → n² (V(i,j)=V(i,j−1)+A[j]) → n log n (best suffix of L ending at m + best prefix of R from m+1; 2T(n/2)+n) → **Kadane** V=max(A[i],V+A[i]), Vmax=max. Max product: track max&min suffix/prefix. Stock buy/sell: D&C with min of left half + max of right half (n log n); or Profit[i]=p[i+1]−p[i] then Kadane O(n); "no way" if best ≤ 0. **Karatsuba** a=2^(n/2)a_1+a_0: ab=2^n a_1b_1+2^(n/2)(a_1b_0+a_0b_1)+a_0b_0; middle = (a_1+a_0)(b_1+b_0) −a_1b_1−a_0b_0 ⇒ 3 mults: 3T(n/2)+n = n^1.585 (4 mults = n²; shifts are ≪). **Strassen** 7 products of n/2 blocks + Θ(n²) adds: 7T(n/2)+n² = n^2.807 (8 → n³). **Closest pair**: split by x, strip 2d sorted by y, ≤7 nbrs each, n log² n (n log n presorted). **Majority / >n/3**: candidates of halves, count in O(n): 2T(n/2)+n = n log n; Boyer-Moore O(n): c=0 adopt, equal c++, else c−−; verify by count. Try-and-check: random pick, count, repeat: E ≤ 2 rounds = O(n). **Grid local min O(n)**: min of border+middle row+col; recurse into quadrant with smaller nbr; T(n/2)+n. **k-sort into k blocks O(n log k)**: median (linear Select) + Partition/Split, recurse both halves, stop at block size n/k. 2T(n/2)+n with T(n/k)=1. **k-near-sorted**: heap of size k+1, O(n log k). **Towers of Hanoi** 2T(n−1)+1 = 2^n−1. Subset sum D&C: 2T(n−1)+1 = 2^n.

A4. Sorting & selection

```
Partition(A,l,r): x=A[r]; i=l-1; for
j=l..r-1: if A[j]≤x: i++,swap(i,j)
  swap(i+1,r); return i+1.  Inv:
A[l..i]≤x | A[i+1..j-1]>x | ? | x
QS: m=Partition; QS(l,m-1); QS(m+1,r).
best 2T(n/2)+n; worst T(n-1)+n=n²
  (sorted input, last pivot); random
pivot: E=Θ(n log n) on EVERY input.
  "average" pivot: T(n/4)+T(3n/4)+n = n
log n.
Select(A,l,r,i): rand pivot→m; k=m-l+1;
i=k→A[m]; i<k→left,i; else right,i-k
Fisher-Yates: for i=1..n swap A[i],
A[Random(1,i)] (uniform 1/n!, O(n))
CountingSort keys 1..k: C[v]++; prefix
sums; for j=n..1: B[C[A[j]]]=A[j];
  C[A[j]]--  (backwards ⇒ stable) Θ(n+k).
Radix: LSD digit by digit,
  stable counting sort each, Θ(d(n+k));
base n ⇒ ints < n^c sort in Θ(cn).
```

QS proof: z_1<…<z_n sorted; z_i,z_j compared ⇔ first pivot from {z_i..z_j} is z_i or z_j ⇒ Pr=2/(j−i+1); E=Σ_{i<j}2/(j−i+1) ≤ Σ_i 2H_n = O(n log n) (row i sums to O(log n)). **Select:** good pivot (middle half) prob ½, removes ≥¼; E[tries per good]=2; size ≤(¾)^i n after i good; E ≤ Σ 2(¾)^i n = 8n = O(n). Indicator version (Tut 4): i≤k≤j: 2/(j−i+1) (diagonals O(1) each); i<j<k: 2/(k−i+1) (columns); k<i<j: 2/(j−k+1) (rows); each sums O(n). Deterministic median (black box) ⇒ T(n/2)+n = O(n); weighted median: partition at median, recurse on heavy side, O(n).

Sort	Worst	Notes	Space	Stable
Insertion	n²	Θ(n+I), I inversions; sorted: n; min comparisons on sorted; k-sorted: O(nk); online	1	Y
Selection / Bubble	n²	always n² / Θ(n) best with early-exit flag	1	N / Y
Merge	n log n	always; recursive; optimal	n	Y
Heap	n log n	always; in place; optimal; not recursive	1	N
Quick(rand)	n²	E n log n; recursive; fastest in practice	log n	N
Counting	n+k	ints in [1,k]	n+k	Y
Radix	d(n+k)	d digits base k	n+k	Y

Radix linear iff w=log U = O(log n) iff range U = poly(n); T=Θ(⌈w/log k⌉(n+k)), best base k=n. [0,n³): 3 passes Θ(n). [1,n^(ln n)]: w=Θ(log²n) ⇒ Θ(n log n), no better than comparison; k=U ⇒ super-poly. 2^Θ(n) range: n²/log n. Fractions p/q, p,q≤n in O(n): gap ≥1/n² ⇒ key K=⌊n²p/q⌋∈[n,n³], radix base n (no gcd, no floats). Strings total length n, variable lengths: bucket by length, LSD from longest, O(n) (padding → Ω(n²); n/2 length-1 + one length-n/2 string). Bucket sort: uniform [0,1), Θ(n) average. Adjacent-swap sorts need Ω(n²) (one inversion per swap, reverse array has n(n−1)/2); long-distance swaps: ⌈n/2⌉.

Pick: ints in [1,n]→counting Θ(n); ints ≤n²→radix base n Θ(n); few inversions (O(√log n)) / each ≤d away→insertion O(n+I) or heap of size d+1 O(n log d); stable n log n→merge; in-place n log n→heap; general floats→merge/heap n log n worst (quick n²). Range queries (x,y] count: sort, two LowerBound → O(log n) after O(n log n); with ints ≤k: counting array C (prefix counts), answer C[y]−C[x] in O(1), build Θ(n+k); range-SUM: C[i]=i·D[i]+C[i−1].

COMP3711 Cheat Sheet v2 — SIDE B: randomization · heaps · lower bounds · past-exam answer bank (11 midterms 2015–2026) · exam protocol

B1. Randomization

Recipe: X=ΣX_i (indicators or waiting times); E[X_i]=Pr[X_i=1] or 1/p; add (**linearity, no independence needed**); simplify. E[XY]=E[X]E[Y] only if independent. Tail: E[X]=Σ_{i≥1}Pr[X≥i]. Geometric p: E=1/p (=Σ j(1−p)^(j−1)p). Measure = expected time on worst-case input (max over I of E_R). Both bounds: max-term up, half-trick down. Dice: E=3.5; max of two=4.47; sum=7.

Classic	Decomposition → answer
Hiring	Pr[i-th hired]=1/i → H_n=Θ(log n); exactly 1 hire 1/n; n hires 1/n!
Hat check / fixed pts / card guessing	each 1/n → E=1; with memory: 1/(n−i+1) → H_n
Hat as good or better (quality 1 best)	Pr=i/n → Σi/n = (n+1)/2
Coupon all n	stage i waits n/(n−i) → nH_n=Θ(n log n)
Coupon first n/2	n(1/n+…+1/(n/2+1)): ≤ n·(n/2)(2/n)=n, ≥ n·(n/2)(1/n)=n/2 → Θ(n)
Coupon last half	n·H_{n/2} = Θ(n log n); last log n: Θ(n loglog n)
Distinct after n boxes	miss type i: (1−1/n)^n → n−n(1−1/n)^n ≈ 0.63n (exact form required)
Empty slots, n into n	n(1−1/n)^n ≈ n/e; exactly-one: n(1−1/n)^(n−1); lonely key: (1−1/n)^(n−1)
Birthday / lockers / pairs	pair collides 1/m → C(n,2)/m = n(n−1)/(2m); ≈1 pair at n≈√(2m)+1 (28 for 365)
Hashing chaining	E[collisions with x] ≤ n/m=α; ops O(1+α); m=Θ(n) ⇒ O(1). Thrown-away hashing: Pr[i-th thrown]=1−((n−1)/n)^(i−1); E = n − n(1−((n−1)/n)^n)
Local minima in random heap-shaped tree (S19)	root 1/3, internal 1/4, leaf 1/2 → 1/3+(n−3)/8+(n+1)/4 = (9n+5)/24
Reservoir k of N	item t kept k/t at step t, k/N final; ever stored k(1+H_N−H_k)
Random-access search (F25)	binary search with random record; good call (25–75%) prob ½ removes ¼; E[calls per good]=2; cost ≤ Σ 2(¾)^i n = 8n = O(n)
Uniform {0,1,2} from fair coin	2 bits, reject 11; E[flips]=2·(1/(3/4))=8/3
Try-and-check majority	success >½ → E[rounds]<2, O(n) exp, O(1) space; worst case unbounded

B2. Heaps

Array H[1..n]: children 2i, 2i+1; parent ⌊i/2⌋; complete, height ⌊log n⌋. **Insert**: append, bubble up O(log n). **Extract-min**: last→root, bubble down (swap with smaller child) O(log n). Find-min O(1). Decrease-key up; increase-key down. Delete arbitrary i: set H[i]=−∞, bubble to root, extract-min (needs position(x) map; update map on every swap). **Build-heap O(n)**: bubble down i=⌊n/2⌋..1; ≤ n/2^(h+1) nodes of height h cost O(h): Σ h·n/2^(h+1) ≤ n·Σ h/2^(h+1) ≤ n (Σ_{k≥1} k x^(k−1)=1/(1−x)², at x=½ gives 4; x¼ ⇒ 1). **F25 bottom-up version**: 2^(m−k) heaps of size 2^k−1 + 2^(m−k−1) new roots → make v_i parent of H_{2i−1},H_{2i}, sift v_i down O(k); total O(k·2^(m−k−1)); Σ_k k2^(m−k−1) = 2^m Σ k/2^(k+1) ≤ n+1 = O(n). **Heapsort** n log n in place, not stable, not recursive. Drawing: after EVERY insert write array + tree (0.5 off per mistake).

d-ary heap: children of i: d(i−1)+2..di+1; parent of j: ⌊(j−2)/d⌋+1; height log_d n; insert O(log_d n); extract O(d log_d n); big d when decrease-keys ≫ extracts (Dijkstra d≈m/n); d=n ⇒ sorted array. **Sliding-window max** (Tut 5): max-heap of (val,idx), lazy-delete expired roots: each item in/out once, but heap may reach Θ(n) (increasing input) ⇒ O(n log n)/O(n); with position map eager delete ⇒ O(n log W)/O(W); monotonic deque O(n) worst (new elem pops smaller from back; pop front if it is A[i−k]; front = max). **F22 update(Q,x) in O(log w)**: y=front; i=position(y); dequeue; H[i]=−∞; bubble to root; extract-min; insert(x); output H[1].

Designs: k-way merge N items: heap of (value, array, index), size ≤k, O(N log k), O(k) space; D&C alternative merges halves: T(k)=2T(k/2)+O(N) = O(N log k). Running median: max-heap low / min-heap high, |low|−|high|∈{0,1}, insert + rebalance O(log n), median O(1). Optimal merge order / Huffman: repeatedly merge two smallest via heap, Θ(n log n). k-th smallest in read-only heap: aux heap seeded root, pop & push children, O(k log k). Report all <x: DFS from root, prune at ≥x, O(k). **Largest ⌈log n⌉ in O(n loglog n)** (S26): split into n/log n blocks of log n, sort each (log n·loglog n each ⇒ n loglog n total); heap of block tops (n/log n inserts = O(n)); extract-max + push next from same block, log n times (O(log² n)). **Min over m sets, findmin O(1), add O(log m + log n_i)** (S25): per-set heap A_i plus top heap H of (min_i, i) with back pointer p_i to its slot in H; on add, insert into A_i (log n_i); if new min, overwrite H[p_i], bubble up, updating p_j on every swap (log m). d-disordered sort: heap of size d+1, O(n log d).

B3. Decision trees & lower bounds

Comparison algorithm = binary tree; worst case = height ≥ **$\log_2$(#outputs)** (binary tree of height h has ≤2^h leaves). **Write this template:** (1) count distinct outputs N; (2) tree needs ≥N leaves; (3) h ≥ log N; (4) simplify. Sorting: n! ⇒ log n! ≥ (n/2)log(n/2) = Ω(n log n). k smallest in order: C(n,k)k! = n(n−1)…(n−k+1) ≥ (n/2)^k ⇒ Ω(k log n) (k≤n/2). n/k sorted-block lists of size k: (k!)^(n/k) ⇒ (n/k)log k! = Ω(n log k) — do NOT argue "each list needs k log k, times n/k". Merge two sorted n-arrays: C(2n,n) ≥ 2^n ⇒ Ω(n). Max/min need n−1; both ≈3n/2. Median of 3 in ≤3 comparisons: A[1]<A[3]? then A[1]<A[2]? then A[2]<A[3]? (leaves 1,2,3 are indices); max of 3 in 2. Min of 4 recursively: a_1:a_2, a_3:a_4, then winners (4 leaves per side). Non-comparison sorts (counting/radix/bucket) escape the bound by never comparing.

B4. Past-exam answer bank (A/B/U rows, with reason)

T_A vs T_B	Ans	T_A vs T_B	Ans
n^1.5 vs n²/√log n	A	n² vs 2T(n/2)+n (=n log n)	B
O(n²) vs Ω(2^n)	A	n²/log n vs n^1.9	B
O(log n) vs Θ(loglog n)	U	10000√n vs 2^n	A
log³n vs ∛n	A	O(√log n) vs Ω(loglog n)	U
O(n²) vs O(n^2.31)	U	5^n vs 7^n	A
2^n vs √(4^n)	U	n²/(log n)^10 vs n^1.5	B
Ω(n³) vs O(n^2.81)	B	2^√n vs 10^∛n	B
n² vs 3^(log₄n)	B	n^1.1 vs n^1.2/(log n)³	A
n^2.1 vs n² log³n	B	O(n²) vs O(n^1.9 log n)	U
n^9 vs 2^√n	A	Ω(n²) vs O(n^1.93)	B
O(n²) vs O(n log n)	U	n vs 3^(log₄n)=n^0.79	B
Ω(n³) vs O(n²)	B	Ω(n/log n) vs Ω(10^10)	U
O(n²) vs Θ(n^2.31)	A	n³ vs O(n^log₂7)	B
Σ_{i≤n} i√n (=n^2.5) vs n²	B	4T(n/4)+n (n log n) vs Σ2^(−i) (Θ(1))	B
O(log(n^10)) vs Ω(n^0.1)	A	n^1.5 log₃n vs log₃(n!) = n log n	B
2^n vs 2^(n/2)	B	O(log^10(log n)) vs Ω(n log n)	A
O(Σ2^(−i)) vs Ω(log n)	A	O(16^(log₄n)+n^1.5)=O(n²) vs 5T(n/2)+n² = n^2.32	A
Σ i√i (n^2.5) vs n^(4/3)	B	Ω(√(5n)) vs Θ(√5^n)	U

Order-the-functions answers: (log n)² < log(2^n) = 2^(log n) = n < n log n+1000 < n^1.1. 10^(10^10) < log⁹n < n < n log n < n^1.1/log n. Tut1: n^(1/log n)=2 < H_n < 2^√log n < n²−n < (log n)^(log n) < n^√log n < 2^(log n)².

Recurrence derivations asked "from first principles": 27T(n/3)+n²: =3^(3k)T(n/3^k)+n²(1+3+…+3^(k−1)) = n³+n²(n−1)/2 = Θ(n³) (ratio a/b^d=3, increasing). 64T(n/4)+n³: level cost 4^(3i)(n/4^i)³ = n³ each ⇒ n³+n³log₄n = Θ(n³ log n). 10T(n/3)+n²: level i cost (10/9)^i n², geometric ↑ ⇒ Θ(n^(log₃10)) = n^2.096 (write 10^(log₃n)=n^(log₃10)). 4T(n/4)+n log₄n (recursion tree): level i: 4^i nodes, each (n/4^i)log₄(n/4^i), level sum n(log₄n − i); leaves 4^(log₄n)=n, cost n; total n + Σ_{i<log₄n} n(log₄n−i) = n + n·log₄n(log₄n+1)/2 = Θ(n log² n). 2T(√n)+log n: n=2^k, S(k)=2S(k/2)+k = k log k ⇒ log n·loglog n. 2T(n−1)+Θ(1) = Θ(2^n). T(n)=T(n/2)+n = 2n−1 exactly.

Design questions that recur (idea + time): Distinct values of sorted array O(log²n) [S25,S26]: LastOcc binary search per block, Θ(log n) blocks. Range query count O(log n) [F22]: sort, Findnext(x) twice, Y−X. Range-sum O(1) [F22]: D[v]=count, C[i]=i·D[i]+C[i−1], answer C[y]−C[x], build Θ(n+k). k blocks O(n log k) [S16,F18,F22]: Select median + Split, recurse, T(n,m)=2T(n/2,m/2)+n, stop at m=1 after log m levels ⇒ n log m. k-way merge [F17]: D&C O(kn log k) or heap O(kn log k). kth of two sorted arrays O(log k) [F17,F25]. Counting-sort range query [F18]: C[i]=#≤i, answer C[y]−C[x]. Sliding-window min with FIFO + heap + position() in O(log w) [F22]. Heap insert sequence drawing [F25 4,3,8,2,5,6,7,9; S26 5,8,1,4,7,3,2,6]. Build heap from pairs of heaps O(n) with Σk/2^(k+1) [F25]. Largest ⌈log n⌉ in n loglog n [S26]. m sets findmin/add [S25]. Random-access search E=O(n) [F25]. Coupon n/2 → Θ(n) with BOTH bounds; distinct after n boxes n−n(1−1/n)^n [S25,S26]. Ω(k log n) for k smallest in order [F25]. Radix pass tables [S19 3-digit, F25 4-digit]: after each pass the rows are sorted by that digit, ties keep previous order. Which sort: ints [1,n] counting O(n); O(√log n) inversions insertion O(n+√log n)=O(n); floats merge/heap O(n log n). Stability: stable = insertion, merge, counting, radix, bubble, selection(as taught); not = quick, heap. Recursive: merge, quick. Min comparisons on sorted input: insertion. Optimal worst: merge, heap. Least space: insertion, heap.

B5. Exam protocol

Time: 120 min for 45–60 pts ⇒ ~2 min per point; do A/B/U + recurrence first (10 min), heap drawing (10 min), then designs. **Every design answer:** (1) one-line idea; (2) "Procedure Name(inputs) → output" with precondition; (3) pseudocode or numbered steps; (4) black boxes named ("Select from class, Θ(n)"; "Merge, Θ(n)"; "binary search O(log n)"); (5) recurrence or sum, solved, final Θ; (6) one line why it is correct. **Running-time sentence:** "Each recursive call does O(f) work and the recursion depth is …, so …". **Expectation:** define X_i in words, state Pr, invoke linearity explicitly, simplify, give exact form when asked (no ≈). **Lower bound:** count outputs, leaves ≥ outputs, height ≥ log. **A/B/U:** write the simplified form next to each (e.g. "=n²") so partial credit is visible. **Heap drawing:** array under each tree. **Never leave blank:** write the recurrence even if unsolved; write the obvious O(n log n) sort-based solution then improve. **Pitfalls:** Θ with α,β when asked; include leaf level; no Master when forbidden; define subroutine I/O; justify every running time; don't assume independence; both bounds when asked; U only when a one-sided bound truly leaves it open; (1−1/n)^n vs ^(n−1) (self excluded ⇒ n−1); log means $\log_2$; "faster" = smaller running time.

Math facts: log_b a = log a/log b = 1/log_a b; a^(log_b c) = c^(log_b a); C(n,k) ≤ n^k/k!, C(2n,n) ≥ 4^n/(2n+1) ≥ 2^n; Stirling n! ≈ √(2πn)(n/e)^n; Σ_{i=1}^{k} r^i = (r^(k+1)−r)/(r−1); 1+2+…+2^(k−1) = 2^k−1; Σ_{i<k} 2^i·(n/2^i)^c for c>1 = Θ(n^c); e^x ≥ 1+x; (1−p)^k ≤ e^(−pk); H_n ≈ ln n + 0.577; $\log_2$10 ≈ 3.32; $\log_2$3 ≈ 1.585; $\log_2$5 ≈ 2.32; $\log_2$7 ≈ 2.807; $\log_3$10 ≈ 2.096; $\log_4$3 ≈ 0.79; √2 ≈ 1.414; 2^10 ≈ 10³.